

# CS120 Computer Network Project Report (TrackMaker-rs)

Zebang He  
2023533189

ShanghaiTech University  
hezebang2023@shanghaitech.edu.cn

Zichen Zhang  
2023533023

ShanghaiTech University  
zhangzch2023@shanghaitech.edu.cn

**Abstract**—TrackMaker-rs is a Rust implementation of a layered communication stack that transmits IP packets over an acoustic channel. The system bridges real-time audio I/O (JACK / PipeWire-JACK) with a custom physical layer (line coding + framing + synchronization), a reliability and medium-access layer (Stop-and-Wait ARQ with CSMA-style carrier sensing/backoff), and a lightweight network layer integration that supports ICMP “ping”, routing/NAT between interfaces, a minimal DNS responder, and a TUN virtual network interface.

The key design goal was to make an “audio modem” that can interoperate with standard OS networking tools and protocols where feasible, while remaining robust to noise and timing drift. TrackMaker-rs exposes this stack via a CLI (transmit/receive, ping/ip-host, router, tun), and uses careful real-time buffer management to operate inside JACK callbacks.

## I. INTRODUCTION

This project follows a course progression from building a reliable link over audio to supporting network-layer packet handling and application-layer services. We implemented:

- A **physical layer** that maps bytes to audio samples using Manchester or 4B5B+NRZI line coding, plus a preamble-based synchronization strategy and CRC8 integrity.
- A **MAC/reliability layer** combining carrier sensing (CSMA-like) with **Stop-and-Wait ARQ** acknowledgements, retransmission, and duplicate suppression.
- A **network-layer toolchain** including an acoustic ICMP ping client/server, a multi-interface router with NAT, and a Linux TUN device integration.
- A **basic DNS service** running inside the router, answering A-record queries from a local table.

The repository structure mirrors this layering: `src/phy/` (encoding/decoding), `src/mac/` (CSMA + acoustic interface abstraction), `src/net/` (tools, router, NAT, fragmentation, tun), and `src/audio/` + `src/device/` for real-time audio transport.

## II. SYSTEM OVERVIEW

### A. High-level architecture

The overall pipeline is:

- 1) **Packet source**: Applications produce payloads and are encapsulated in frames.
- 2) **MAC / reliability** (`src/mac/`): frames are queued, transmitted with CSMA-style sensing/backoff, and acknowledged using Stop-and-Wait ARQ.

- 3) **PHY encoding** (`src/phy/`): frames are converted to a waveform: preamble + sync pattern + line-coded bits.
- 4) **Audio device** (`src/audio/`, `src/device/`): samples are sent/received via JACK process callbacks using shared ring buffers.
- 5) **PHY decoding**: receiver correlates preamble, aligns to the payload boundary, decodes line coding, validates CRC, and emits frames upward.

### B. Real-time audio transport

Audio I/O is handled with a JACK client. A process callback reads input samples into a shared record buffer and writes output samples from a playback buffer. The application coordinates Recording/Playing modes through an AppState state machine. This structure allows the PHY to run concurrently while respecting real-time constraints (no blocking operations inside the callback).

Operationally, we found buffer/quantum configuration affects reliability. After testing on different configuration groups, we recommend PipeWire settings (48 kHz, 128–256 quantum) and practical volume guidance to reduce distortion/oversampling artifacts.

## III. ENVIRONMENT, CLI, AND AUDIO INTERFACE

### A. CLI entry points

TrackMaker-rs provides a multi-command CLI including:

- `tx / rx`: file transfer over the acoustic link.
- `ping / ip-host`: ICMP echo request/reply over audio.
- `router`: multi-interface router with NAT and DNS.
- `tun`: TUN forwarding mode to present the acoustic channel as a virtual interface.

The router mode also documents how to run with Linux capabilities (`setcap`) rather than `sudo`, since JACK typically runs under the user session.

### B. Practical audio setup

The README includes both macOS JACK setup (CoreAudio + jackd options) and Linux PipeWire-JACK tuning. These recommendations were incorporated because acoustic networking is sensitive to sampling rate mismatches, buffer jitter, and nonlinear distortion.

## IV. PROJECT 1: PHYSICAL LAYER (PHY)

### A. Frame format and integrity

We implement a compact frame format:

- **Length** (2 bytes)
- **CRC8** (1 byte)
- **Type** (1 byte)
- **Sequence** (1 byte)
- **Source ID** (1 byte)
- **Destination ID** (1 byte)
- **Payload** (variable)

CRC8 is computed using polynomial  $0x07$ . Frames failing CRC are discarded early. Destination filtering is also applied so nodes can ignore frames not addressed to them.

### B. Line coding

Two line-coding options are supported:

- **Manchester** coding (self-clocking, robust at the cost of  $2\times$  symbol transitions).
- **4B5B + NRZI** coding (improves transition density compared to raw NRZ while being more bandwidth-efficient than Manchester).

The line coding choice is configurable at runtime. This allows experimenting with robustness vs throughput trade-offs.

### C. Synchronization and decoding

A preamble-based synchronizer detects frame start in the received sample stream using correlation. After initial detection, the decoder refines alignment using a sync pattern so the bit boundary is stable even under noise and minor drift.

For performance, the correlation uses optimized dot-product paths (including SIMD/AVX when available). This helps keep decoding real-time at 48 kHz.

### D. Design rationale

- **Correlation sync** is resilient to noise and amplitude changes compared to simple threshold triggering.
- **CRC8** is a cheap integrity check suitable for short acoustic frames.
- Supporting both Manchester and 4B5B+NRZI made it easier to debug and compare failure modes.

## V. PROJECT 2: MAC LAYER (CSMA + RELIABILITY)

### A. Channel sensing and backoff

We implement CSMA-like behavior by measuring channel “busy” state (energy threshold). Before transmitting, the node waits for a clear channel and then uses randomized backoff (contention window) to reduce collisions.

### B. Stop-and-Wait ARQ with ACK frames

Reliability is provided by Stop-and-Wait:

- Each data frame carries a sequence number.
- The receiver sends an ACK with the sequence number.
- The sender retransmits on timeout.
- The receiver suppresses duplicates (but can re-ACK to help recovery).

This design is simple yet effective for an acoustic channel where propagation and processing delays are large compared to packet sizes.

### C. Acoustic interface abstraction

`src/mac/acoustic_interface.rs` exposes a higher-level “send/receive packet” API to the network layer. It also integrates IP fragmentation/reassembly logic so the acoustic MTU can remain small without preventing IP-level traffic.

## VI. PROJECT 3: NETWORK LAYER INTEGRATION (ICMP, ROUTING, NAT, TUN)

### A. ICMP ping tools

The ping tool constructs IPv4 + ICMP echo requests and reports RTT statistics (similar to standard ping). The `ip-host` tool responds to echo requests.

This provides an end-to-end validation path spanning the entire stack: packet construction  $\rightarrow$  MAC/PHY  $\rightarrow$  audio channel  $\rightarrow$  decode  $\rightarrow$  reply  $\rightarrow$  statistics.

### B. IP fragmentation and reassembly

Because the acoustic frame payload is limited, TrackMakers includes an IPv4 fragmentation module that splits packets based on MTU and reassembles them on receive. Fragment offsets follow the IPv4 8-byte unit rule.

This allows larger IP packets to traverse the acoustic link while keeping the PHY frame size manageable for reliability.

### C. Multi-interface router

The router captures packets from multiple interfaces:

- **Acoustic interface** (the audio link)
- **WiFi interface** (pcap capture/inject)
- **Ethernet/gateway interface** (pcap capture/inject)
- **TUN interface** (for OS integration)

It parses IPv4 headers, decrements TTL for forwarded packets, and sends frames out the appropriate interface based on a routing table.

### D. NAT

The router supports NAT in two forms:

- 1) **ICMP NAT** using an identifier mapping table (for echo requests/replies).
- 2) **TCP/UDP “session” NAT** using a simple mapping from external port  $\rightarrow$  internal IP (cone-style). When inbound traffic targets the router’s WAN IP, the router checks whether the destination port matches an existing internal mapping and rewrites the destination IP accordingly.

After address translation, the router recalculates:

- the IPv4 header checksum
- the TCP/UDP checksum (pseudo-header + L4 header + payload)

This is critical for correctness when rewriting addresses.

### E. TUN device

The tun mode creates a virtual interface (e.g., tun0) and forwards packets between the OS and the acoustic interface.

- Outbound packets from the OS are routed to a destination MAC based on whether the destination is in the local subnet or should go through a configured gateway.
- Inbound packets received from audio have their IPv4 checksums corrected before being written into the TUN device.

This feature makes the acoustic link usable by standard IP applications without modifying the OS network stack.

## VII. PROJECT 4: DNS AND APPLICATION-LAYER NOTES

### A. DNS service in router

The router includes a minimal DNS responder:

- Detects UDP packets with destination port **53**.
- Parses a single-question DNS query (without compression pointer support in the request name parsing).
- Supports **A records (Type 1) in IN class (1)**.
- Resolves from a local table (router.lan, node1.lan, node3.lan, plus a few hardcoded external examples).
- Returns either a normal answer or **NXDOMAIN**.

This DNS implementation is intentionally small and tailored to typical local queries.

### B. HTTP

We did not implement a dedicated HTTP client/server in the codebase. However, because the router supports IPv4 forwarding and TCP/UDP NAT behavior, application-layer protocols such as HTTP can traverse the system if endpoints exist and the traffic fits within the constraints of the acoustic link (latency, throughput, MTU/fragmentation).

## VIII. EVALUATION AND DISCUSSION

### A. Throughput and overhead

The PHY configuration targets a nominal bitrate of 12 kbps at 48 kHz sampling. Effective application throughput is lower due to:

- line-coding overhead (Manchester doubles transitions; 4B5B adds 25% symbols)
- framing overhead (headers + CRC + preamble)
- CSMA backoff and channel idle time
- Stop-and-Wait ACK and retransmission
- fragmentation overhead for large packets

In practice, these mechanisms trade raw throughput for stability and correctness on a noisy acoustic channel.

### B. Robustness

Key robustness features include:

- correlation-based synchronization
- CRC validation and destination filtering
- retransmission and duplicate suppression
- conservative frame sizing to reduce error probability

### C. Implementation challenges

- **Real-time constraints:** decoding/encoding must not block JACK callbacks.
- **Audio non-idealities:** clipping, oversampling, and noise can corrupt symbol timing.
- **Checksum correctness:** NAT requires recomputing L3 and L4 checksums; missing this breaks interoperability.
- **Interfacing with OS/network devices:** using pcap + TUN requires careful packet parsing and privilege handling.

## IX. CONCLUSION

TrackMaker-rs demonstrates a complete, layered networking system operating over sound: from sample-level PHY up through packet-level routing, NAT, and DNS, with OS integration via TUN. The design emphasizes correctness and robustness under real-world audio constraints, while still exposing familiar networking workflows (ICMP ping, routing, NAT, DNS resolution).

## APPENDIX

### How to Run (quick reference)

- Acoustic ping client: `cargo run -- ping 1.1.1.1 --gateway <gw> --local-ip <acoustic-ip>`
- Acoustic ping host: `cargo run -- ip-host --local-ip <acoustic-ip>`
- Router (PipeWire-JACK recommended): see README for interface/IP/MAC environment setup.